

字符串类

字符常量类 String 的内存模型

字符串常量，是存在**常量池**里，这个常量池不负责存储程序员声明的其他static final这种常量；
(常量池是堆的一部分)

```
// 当声明
String tmp1 = "abc";
// 首先，会去常量池里找是否存在"abc"对象
// 如果没有，则会在常量池里创建"abc"对象
// 如果此时，再声明
String tmp2 = "abc";
// 由于常量池里有"abc"
// 则tmp2直接使用这块内存
// 此时，tmp1 == tmp2 为 true

String tmp3 = new String("abc");
// 这行会在堆里创建一个tmp3对象
// 此时，堆里有一个“abc”对象，常量池里也有一个“abc”对象
// 这两个对象，在内容上是一样的，但是在内存中并不是同一个

// 注意：
String tmp4 = new String("xyz"); // 在内存中会产生几个对象？
// 2个！
// 常量池里有一个；堆里有一个
```

字符串常量存在一个问题：拼接非常耗时，因为常量们分散在内存中，并且拼接之后会产生一个新的常量....

字符串变量类 StringBuffer & StringBuilder

这两东西最大的作用是，用来做字符串拼接；

这两个东西的用法也是完全一致的；

StringBuffer是线程安全的，StringBuilder是线程不安全

String的内置方法

利用字符数组构造字符串

```
char[] c = {'H','e','l','l','o',' ','W','o','r','l','d'};  
String str = new String(c);
```

* 将字符串还原为字符数组

```
char[] tmpC = str.toCharArray();  
System.out.println(Arrays.toString(tmpC));
```

* 字符串截取

```
String s1 = str.substring(5,10); //截取str的[5,10)  
String s2 = str.substring(4); //截取str的[4,结束]
```

获得某个位置的字符

```
char tmpChar = str.charAt(0); //取index=0的字符
```

字符串长度

```
int lenOfStr = str.length();
```

* 查找子串位置

```
int posi = str.indexOf("World"); // 返回5
int posi = str.indexOf("o"); // 查找o第一次出现的位置
int posi = str.indexOf("o", 6); // 从下标6开始，第一次出现o的位置（包含6）
int posi = str.indexOf("xyz"); // 如果不存在子串，返回-1

int posi = str.lastIndexOf("o"); // 最后一次出现o的位置

boolean res = str.startsWith("Hello"); // str是否以Hello开头？
boolean res = str.endsWith("World"); // str是否以World结尾？
boolean res = str.contains("ow"); // str是否包含ow
```

* 字符串判空

```
String str1 = "";
String str2 = "   ";

boolean res = str1.isEmpty(); // 判断是否为空字符串（长度为0）
boolean res = str1.isBlank(); // 不再粗暴的通过长度判断，如果字符串中全是空白字符（空格,制表符...）也会返回true
```

大小写转换

```
String lower = str.toLowerCase();  
String upper = str.toUpperCase();
```

字符串替换

```
String replacedStr = str.replaceAll("l","L");// 也可以替换字符串
```

trim()去掉首尾空白字符

```
String str3 = new String("  aaa  ");  
String tmpStr = str3.trim();// tmpStr是"aaa"
```

日期类

有时区的时间 模型

时间的本质是一个long型数字

```
Date now = new Date();// 获取当前的时间对象
System.out.println(now);// Tue Sep 15 17:51:01 CST 2026 输出
long time = now.getTime();// 通过对时间对象调用getTime(), 可以获得一个1970-1-1-0点到现在的ms数

// 还可以通过long数字设置一个Date对象
long timeMs = 0;
Date org = new Date();
date.setTime(timeMs);

// Date提供了比较方法
System.out.println(now.before(org)); // 判断now是否在org之前
System.out.println(now.after(org)); // 判断now是否在org之后
System.out.println(now.equals(org)); // 判断now是否等于org
```

世俗意义上的日期类 & 数据库中的日期类

```
// java中有两个Date类
// 一个是平常所说的时间
java.util.Date;
// 还有一个是给数据库用的时间
java.sql.Date;
```

日历类 -- 日期设定类

java中的日历类是**Calender**

获取今天的日历

```
Calendar cal = Calendar.getInstance();//表述当前时刻的日历实例
System.out.println(cal);
// 打印结果
/*java.util.GregorianCalendar[
```


设置日历

```
cal.set(Calendar.YEAR, 2026); // 设置年为2020年
cal.set(Calendar.MONTH, Calendar.FEBRUARY); // 设置8月
cal.set(Calendar.DAY_OF_MONTH, 32); // 设置日期是1
cal.set(Calendar.HOUR_OF_DAY, 15); // 小时
cal.set(Calendar.MINUTE, 0); // 分
cal.set(Calendar.SECOND, 0); // 秒
cal.set(Calendar.MILLISECOND, 0); // 毫秒

// 有快速用法
cal.set(2020, Calendar.AUGUST, 1); // 设置年月日
cal.set(2020, Calendar.AUGUST, 1, 15, 0, 0); // 设置年月日 时分秒毫秒
```

使用get获取日历的某个字段

```
int date = cal.get(Calendar.DAY_OF_MONTH); // 获取日期（多少号）
System.out.println(date);
```

Calender与long变量之间的转换

```
//Calendar -> long
long x1 = cal.getTimeInMillis();

//long -> Calendar
cal.setTimeInMillis(x1);
```

Calendar与Date之间的转换

```
//Calendar -> Date
Date d1 = cal.getTime();

//Date -> Calendar
cal.setTime(d1);
```

计算100天后\前的日期

```
//100天后
cal.add(Calendar.DAY_OF_MONTH, 100); // +100天

//100天之前
cal.add(Calendar.DAY_OF_MONTH, -100); // -100天
```

demo：周二执行一次任务

需求：

1. 如果软件的启动时间，在本周二之前，则本周二执行任务
2. 如果软件的启动时间，在本周二之后，则下周二执行任务

```
public static void main(String[] args) {
    Calendar cal = Calendar.getInstance();
    cal.set(Calendar.DAY_OF_WEEK, Calendar.TUESDAY); //本周二
    cal.set(Calendar.HOUR_OF_DAY, 2);
    cal.set(Calendar.MINUTE, 0);
    cal.set(Calendar.MILLISECOND, 0);
    //得到本周二的时间
    Date tue = cal.getTime();

    //当前时间
    Date now = new Date();
```

```
if(now.after(tue)){
    //下一周的周二
    cal.add(Calendar.DAY_OF_MONTH, 7);
    tue = cal.getTime();
}
System.out.println(tue);
}
```

* SimpleDateFormat 日期 - 字符串 转换

SimpleDateFormat是一个类，用于进行时间转换；

```
// String -> Date
String str = "2026-9-15 12:08:44";
//创建转换工具对象，约定日期格式
SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
try
{
    Date date = sdf.parse(str);
    System.out.println(date);
    System.out.println(date.getTime());
}
catch (ParseException e)
{
    System.out.println("字符串格式不符合约定格式");
}

// Date -> String
Date now = new Date();
SimpleDateFormat sdf2 = new SimpleDateFormat("yyyy年MM月dd日 HH时mm分ss秒");
String s = sdf2.format(now);
```

```
System.out.println(s);
```

年 yyyy

月 MM

日 dd (DDD是今年的第多少天)

时 HH (hh是12小时制，HH是24小时制)

分 mm

秒 ss

毫秒 SS

无时区概念的日期类和时间类

无时间概念的类有2个

1. LocalDate

2. LocalDateTime

```
LocalDate date = LocalDate.now(); // 当前日期（不含时分秒）
LocalDateTime time = LocalDateTime.now(); // 当前日期 含有时分秒

// 手动构建时间
LocalDate date = LocalDate.of(2026,1,1);
// 通过LocalDate获取星期
DayOfWeek x = date.getDayOfWeek();
int week = x.getValue();

// 日期的加减
date = date.plusDays(100); // +100天
date = date.plusDays(-100); // -100天
date = date.plusWeek(1); // 往后一周
date = date.minusMonths(1); // 往后一个月
```

